

1일차 · 회귀분석

선형회귀모델

최소제곱 · 경사하강법 · 모델 평가 · 규제 · 다항식 특성

실습: 거시·금융 지표 29개로 코스피지수 예측 (Python · scikit-learn · Colab)

이기홍 교수

오늘의 세 가지 메시지

강의가 끝나면 이 세 문장을 설명할 수 있어야 합니다

1

적합 — 제공오차를 최소화한다

정규방정식(한 번에 계산) 또는 경사하강법(반복 계산): 답은 같고 비용이 다르다.

2

평가 — 표본 밖에서, 정직하게

모델이 보지 못한 데이터로 RMSE · MAE · R^2 를 잰다.

3

통제 — 복잡도를 데이터에 맞춘다

릿지 · 라쏘 · 엘라스틱넷 · 다항식 특성은 편향을 조금 받아들이고 분산을 크게 줄인다.

선형모델은 적합하기도 쉽고 속기도 쉽다. 무엇을 배우는지는 평가 설계가 결정한다.

로드맵

여섯 개 섹션 — 이론을 먼저 보고, 같은 개념을 실제 시장 데이터에 적용한다

#	섹션	핵심 질문
01	선형회귀 기초	어떤 직선이 가장 잘 맞고, 어떻게 찾는가?
02	모델 평가	모델은 얼마나 틀리는가 — 무엇과 비교해서?
03	기초 실습	scikit-learn의 4단계 패턴
04	규제(Regularization)	모델이 노이즈를 외우지 않게 하려면?
05	다항식 회귀	선형모델로 곡선을 잡을 수 있는가?
06	코스피 사례 실습	실제 데이터에서 전체 작업 흐름은 어떻게 되는가?

01

SECTION 01

선형회귀 기초

모델 · 비용함수 · 정규방정식 · 경사하강법

먼저 용어부터

이후 모든 슬라이드에 나오는 말들입니다 — 오른쪽 열은 scikit-learn에서의 이름

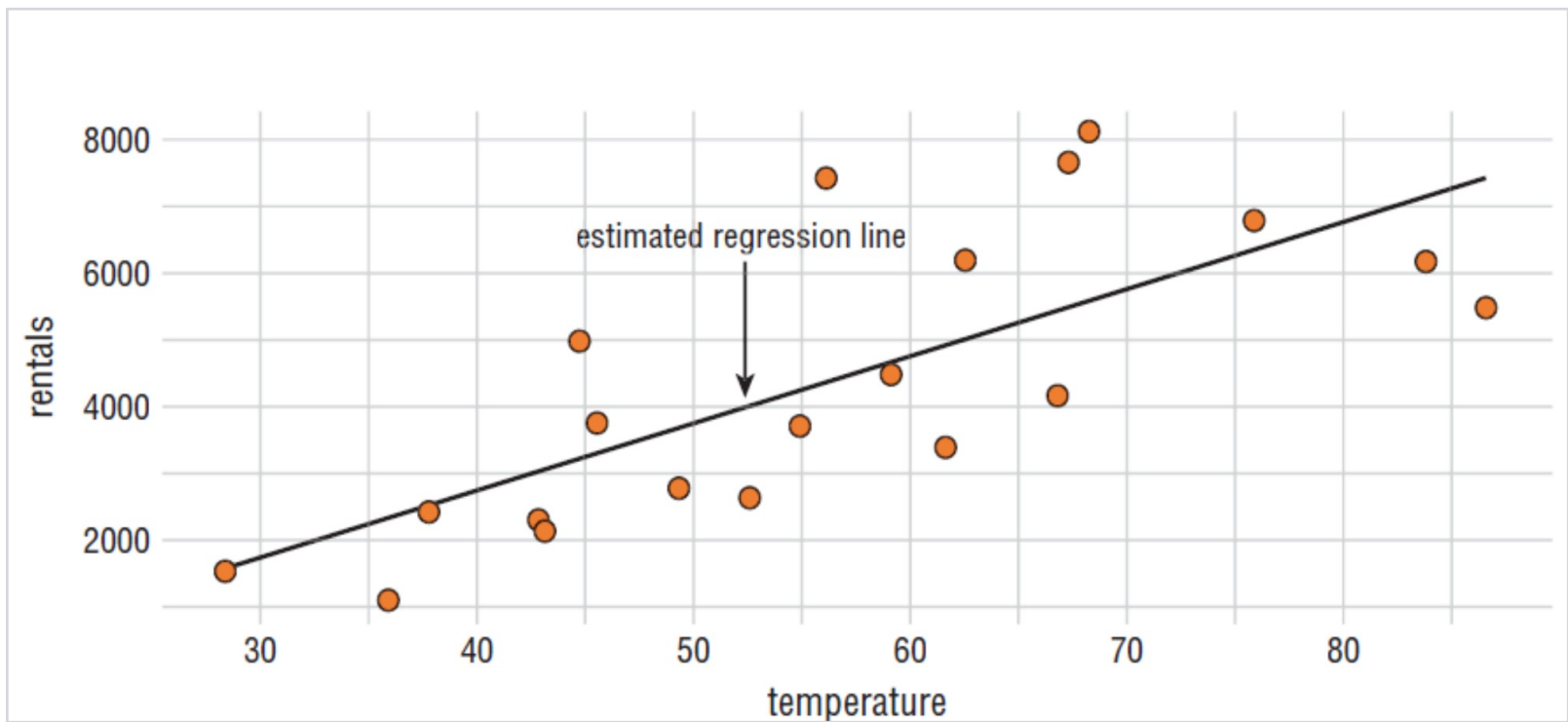
용어	뜻	코드
목표변수 y (종속변수, target)	예측하려는 값	<code>y, y_train</code>
특성 x (독립변수, feature)	y 를 예측하는 데 쓰는 입력	<code>X, X_train</code>
가중치 · 회귀계수 w	다른 특성을 고정하고 x 가 1 늘 때 $\hat{y}$ 의 변화	<code>model.coef_</code>
절편 · 편향 b (intercept, bias)	모든 특성이 0일 때의 $\hat{y}$	<code>model.intercept_</code>
예측값 $\hat{y}$	주어진 x 에 대한 모델 출력	<code>model.predict(X)</code>
잔차 $e = y - \hat{y}$ (residual)	실제값 - 적합값 (계산 가능)	<code>y_test - y_pred</code>
오차 ε (error)	실제 생성 과정의 관측 불가능한 노이즈	—

단순 선형회귀

독립변수 x 하나와 종속변수 y 의 관계를 가장 잘 설명하는 직선(회귀선)을 찾는다

그림 읽기

- y : 자전거 대여량
- x : 기온
- 점 하나 = 하루
- 직선: 회귀선

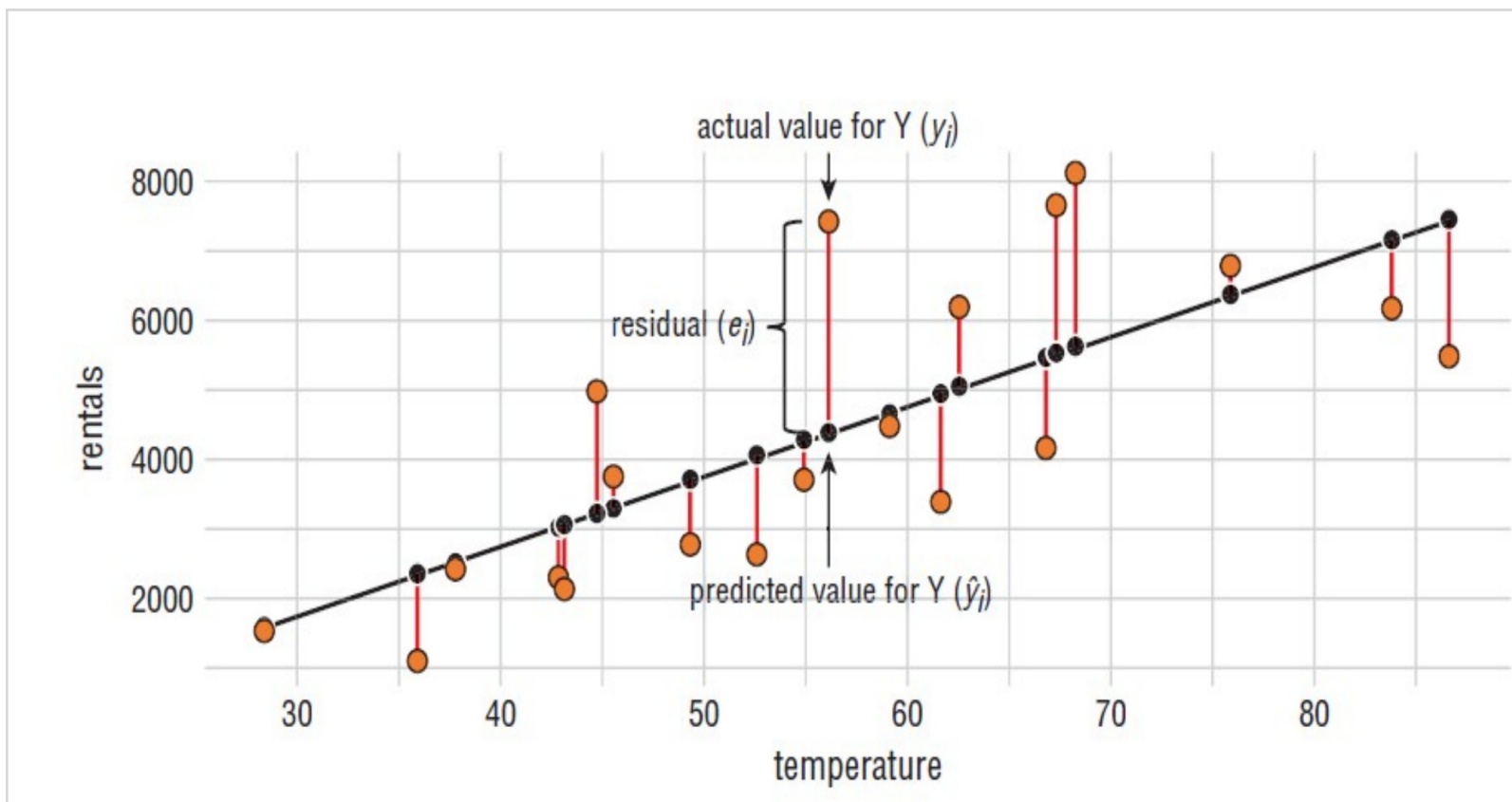


잔차와 최소제곱

잔차 = 실제값 - 예측값. OLS는 잔차 제곱합이 가장 작은 직선을 고른다

그림 읽기

- 주황 점: 실제값 y
- 검은 점: 예측값 $\hat{y}$
- 빨간 막대: 잔차 e
- 제공해서 모두 더한다



선형회귀식

특성 하나에서 n개로, 그리고 벡터 표기로

단순 회귀

$$\hat{y} = w x + b$$

다중 회귀

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

벡터 표기

$$\hat{y} = \sum_{j=0}^n w_j x_j = \mathbf{w}'\mathbf{x} \quad (x_0 = 1, w_0 = b)$$

읽는 법

- w : 기울기 — 각 x 가 $\hat{y}$ 에 주는 효과
- b : 절편 — 모든 x 가 0일 때의 $\hat{y}$

원본에서 고치기 점

- 절편이 두 번 들어가 있었다($b_0 \cdot x_0 \dots + b_0$).
' $+ b$ '와 ' $x_0 = 1$ ' 중 하나만 쓴다.

비용함수: 평균제곱오차(MSE)

손실함수 · 목적함수 · 오차함수라고도 부른다 — 알고리즘이 줄이려는 단 하나의 숫자

$$J(w, b) = \underbrace{\frac{1}{2}}_{\text{미분식을 깔끔하게}} \underbrace{\frac{1}{m}}_{\text{훈련 샘플 } m \text{ 개의 평균}} \sum_{i=1}^m \underbrace{(y_i - \hat{y}_i)^2}_{\text{샘플 } i \text{ 의 제곱오차}}$$

왜 제곱하는가?

- +10과 -10의 오차가 서로 상쇄되지 않는다
- 매끄러워서 미분하기 쉽다
- 큰 오차에 훨씬 큰 벌점 (이상치에 민감)

1/(2m)에 대해

- 엄밀한 MSE는 1/m을 쓴다
- 1/2은 미분할 때 나오는 2를 없애는 용도일 뿐
- 어느 쪽이든 최적해 w^*, b^* 는 같다

해법 1: 정규방정식

비용함수의 미분을 0으로 놓고 행렬 연산으로 한 번에 푼다

① 비용함수를 미분한다

$$\frac{\partial J}{\partial w} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i) x_i$$

$$\frac{\partial J}{\partial b} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)$$

② 둘 다 0으로 놓고 모든 계수를 한 번에 푼다

$$\hat{\mathbf{w}} = (X'X)^{-1} X'y$$

한 번에 정확한 답이 나온다. 다만 특성이 아주 많거나 서로 강하게 상관되면 역행렬 계산이 비싸고 불안정해진다.

작동 방식

- X: b를 위한 1의 열을 붙인 $m \times (n+1)$ 행렬
- 선형방정식 한 번 — 학습률도 반복도 없다
- sklearn LinearRegression은 같은 최소제곱 문제를 안정적인 SVD 방식으로 푼다

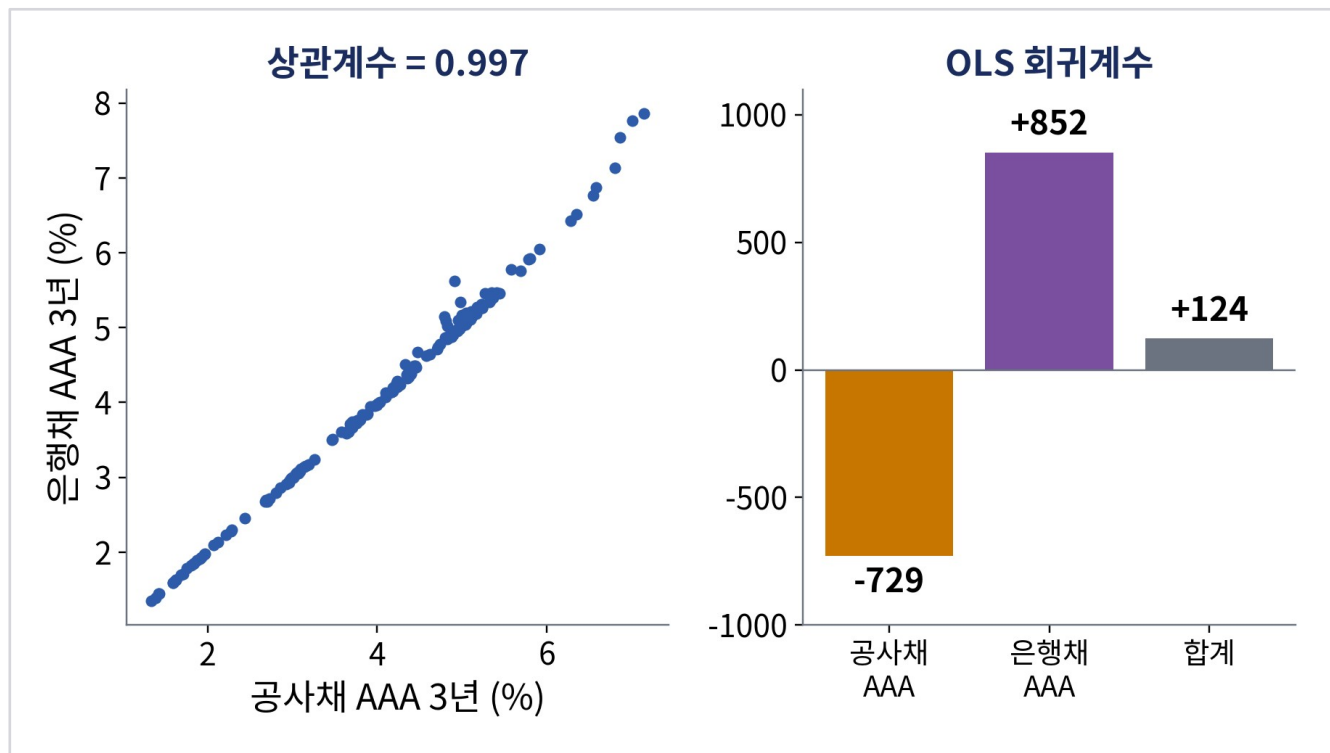
함정: 다중공선성

특성끼리 강하게 상관되면 $X'X$ 가 특이행렬에 가까워져 계수 추정이 불안정해진다

$$\text{Var}(\hat{\mathbf{w}}) = \sigma^2 (X'X)^{-1}$$

무슨 일이 생기나

- $\det(X'X) \rightarrow 0$ 이면 역행렬과 $\hat{\mathbf{w}}$ 의 분산이 폭발한다
- 계수가 크게 흔들리고 부호가 뒤집힌다. 예측은 멀쩡해 보일 수 있다
- 대책: 변수 제거·통합, PCA, 릿지



코스피 실습 데이터: AAA 3년물 금리 두 개(상관 0.997)의 계수가 -729와 +852 — 합은 +124뿐이다.

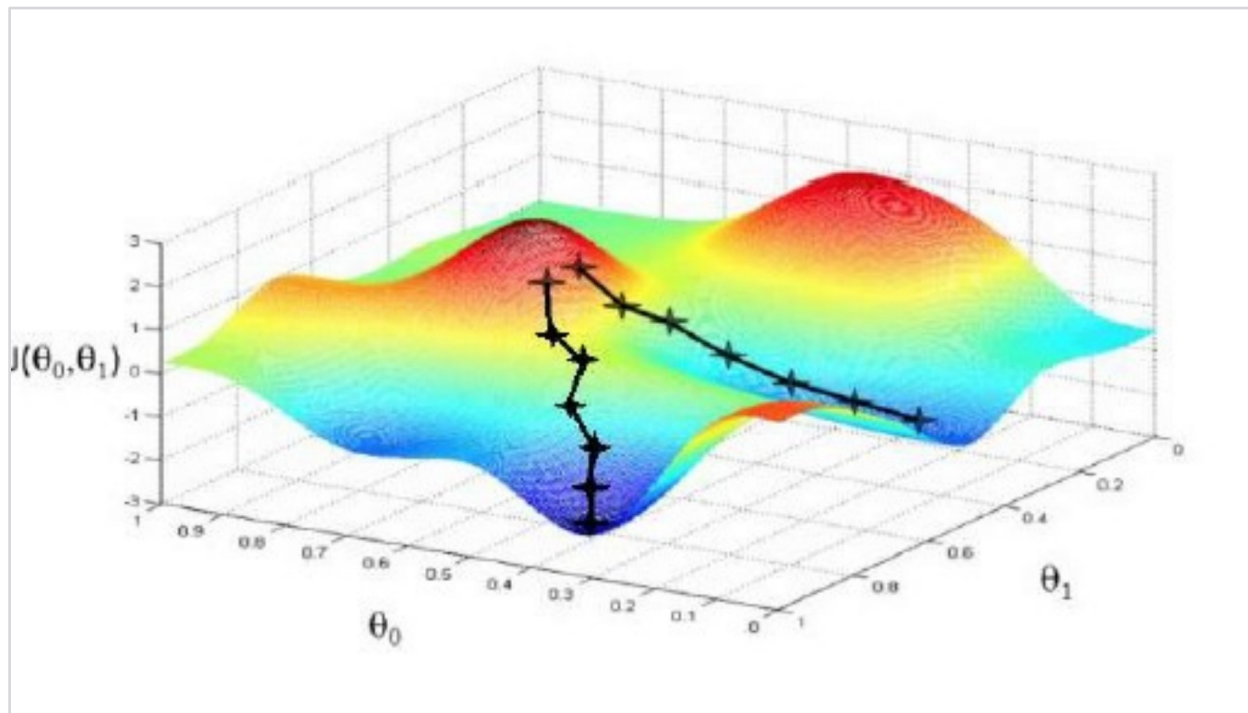
해법 2: 경사하강법

아무 데서나 시작해 기울기의 반대 방향으로 한 걸음씩, 비용이 더 줄지 않을 때까지 반복한다

$$w \leftarrow w - \eta \frac{\partial J}{\partial w} \quad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

핵심 아이디어

- η (에타) = 학습률 = 보폭
- 기울기는 오르막 방향 $\rightarrow$ 반대로 간다
- J 가 거의 변하지 않으면 멈춘다 (수렴)
- 그림의 곡면에는 지역 최솟값이 있지만, OLS 비용함수는 전역 최솟값 하나인 볼록 함수다

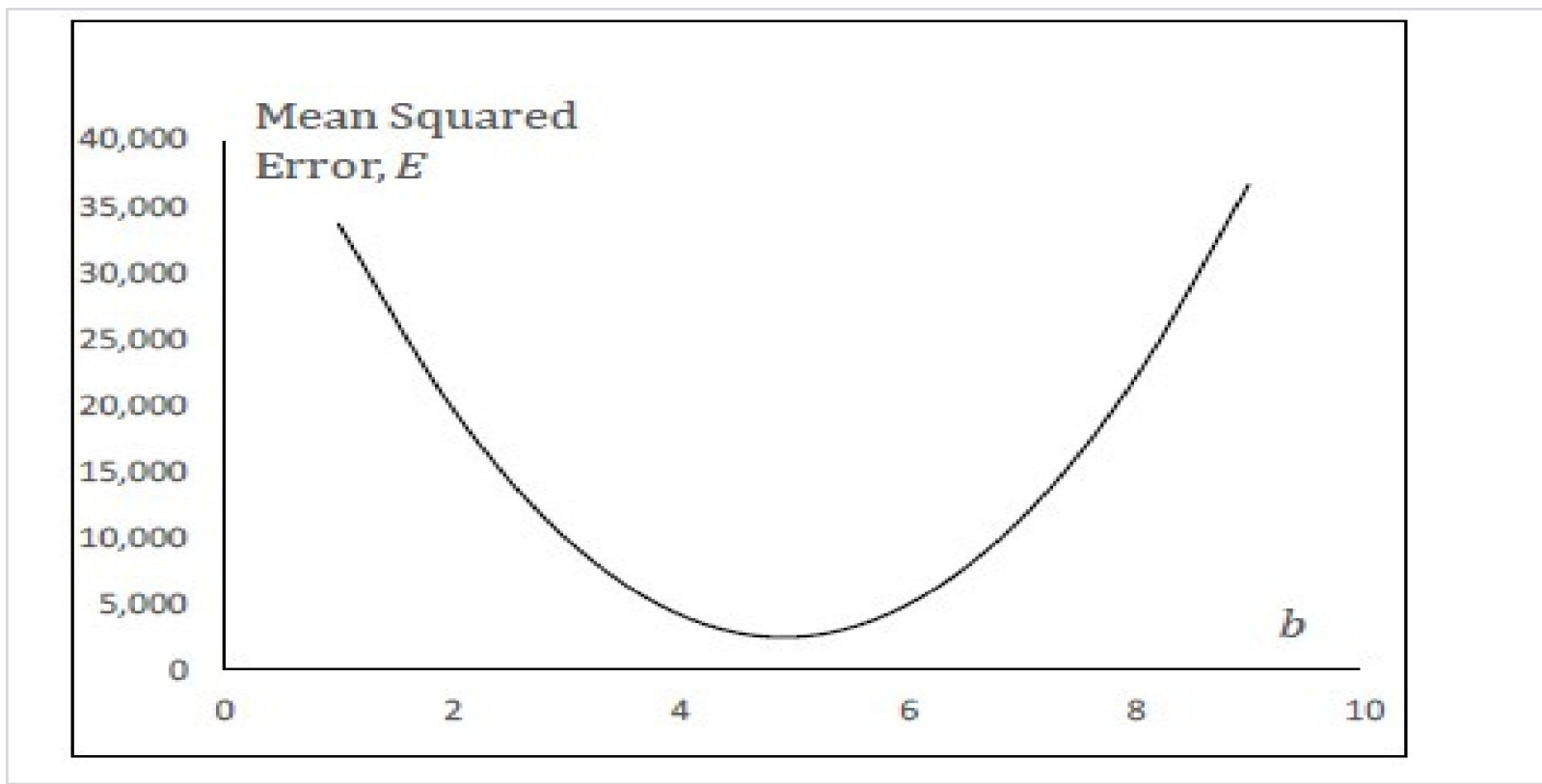


경사하강법 예시: 계수 하나

w 를 고정하면 절편 b 에 대한 MSE는 포물선이다 — 경사하강법은 그 바닥까지 내려간다

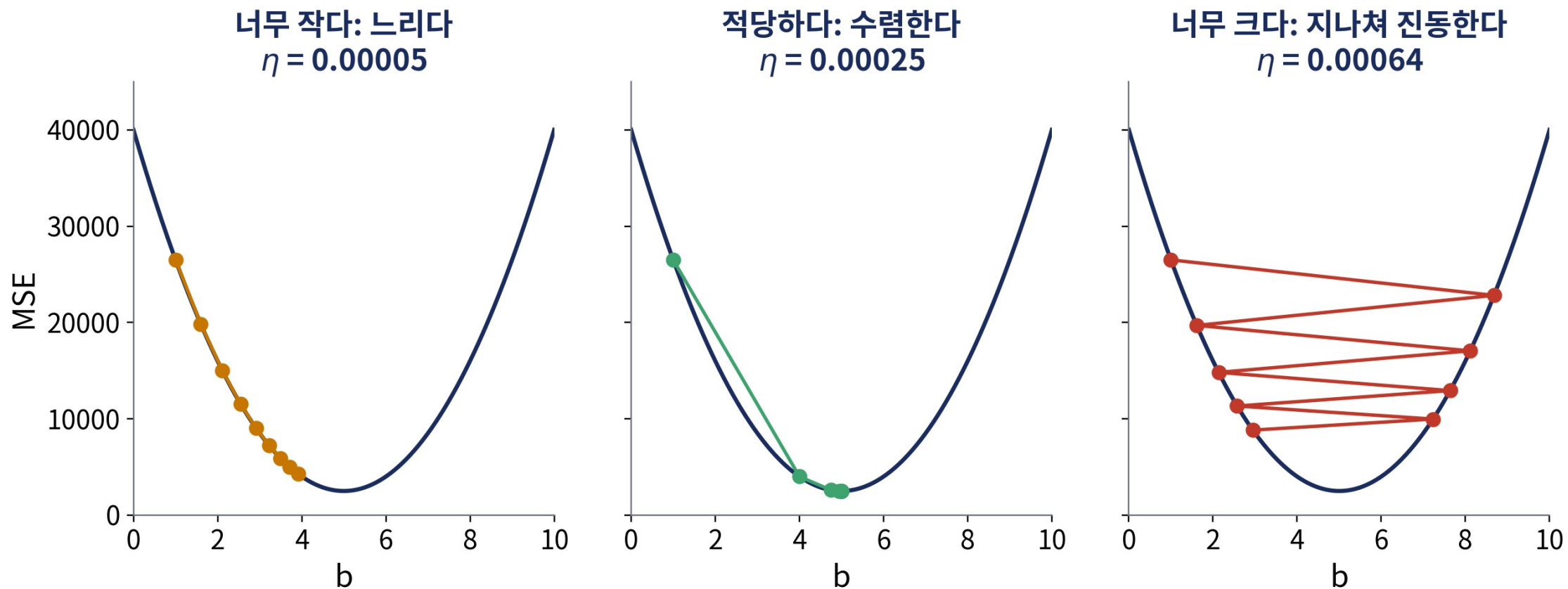
그림 읽기

- 가로축: b
- 세로축: MSE
- 최솟값 $b \approx 5$
- 기울기 부호 = 방향



학습률 η 고르기

너무 작으면 한없이 느리고, 적당하면 몇 걸음에 수렴하고, 너무 크면 지나쳐서 진동하거나 발산한다



정규방정식 vs. 경사하강법

같은 최소제곱 해 — 데이터 크기와 유연성으로 고른다

구분	정규방정식	경사하강법
해	한 번에 정확히	반복해서 근사
정할 설정값	없음	학습률 η , 반복 횟수
특성 스케일링	필요 없음	사실상 필수
특성 수 n 이 클 때	계산량 $\sim n^3$ — 매우 느림	한 걸음당 $\sim m \cdot n$ — 잘 확장됨
데이터 m 이 크거나 계속 들어올 때	전체를 한 번에 메모리에	미니배치 · 확률적 경사하강법 (SGD)
scikit-learn	LinearRegression	SGDRegressor

02

SECTION 02

회귀모델 평가

RMSE · MAE · MAPE · R^2 — 그리고 각 지표가 감추는 것

지표보다 먼저: 어디서 재는가?

지표는 그것을 계산한 데이터만큼만 정직하다

훈련 세트 (표본 내, in-sample)

- w 와 b 를 적합하는 데 쓴다
- 이미 본 데이터라 오차가 낙관적이다

테스트 세트 (표본 외, out-of-sample)

- 적합하는 동안 전혀 손대지 않는다
- 실제 성과를 추정하는 오차

**원칙: 모델도 스케일러도 훈련 세트에서만 fit한다.
테스트 세트는 맨 마지막에 한 번만 쓴다.**

α 같은 설정값을 조정하려면 훈련 데이터에서 검증 세트를 떼어 내거나 교차검증을 쓴다.

예측력: RMSE, MAE, MAPE

RMSE는 가장 널리 쓰는 예측 지표지만 '평균 오차'는 아니다 — 그건 MAE다

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (\%)$$

RMSE

- y와 같은 단위
- 먼저 제곱하므로 큰 오차가 좌우한다
- 항상 $\text{RMSE} \geq \text{MAE}$

MAE

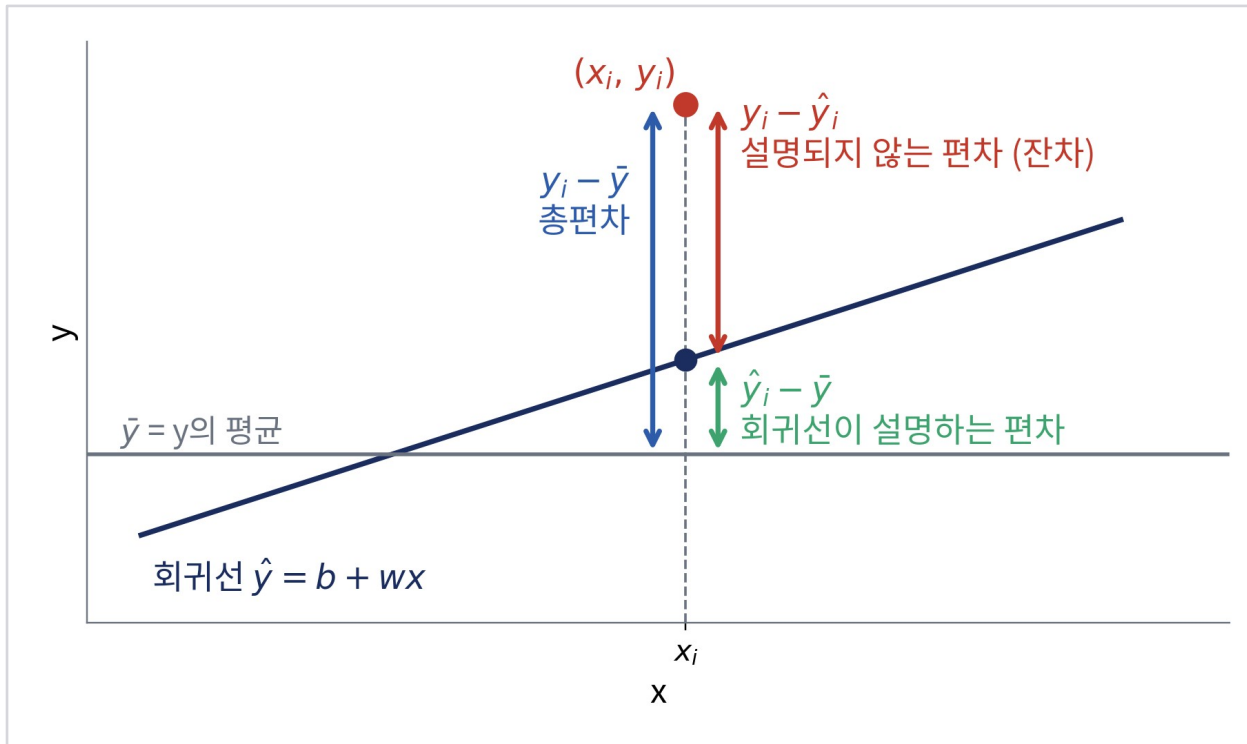
- 평균 절대 오차
- '대략 이만큼 틀린다'는 해석은 여기에 맞다
- 이상치에 덜 민감

MAPE

- 단위 없는 % 지표
- $y = 0$ 이면 정의되지 않는다
- y가 0에 가까우면 폭발한다

설명력: R^2

y의 변동 가운데 모델이 설명하는 비율



$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{SST (총변동)}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{SSR (설명된 변동)}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{SSE (잔차 변동)}}$$

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = \frac{\text{SSR}}{\text{SST}}$$

의미

- $R^2 = 1$: 변동을 모두 설명
- $R^2 = 0$: y의 평균으로 예측한 것과 같음

R²: 공식이 말해 주지 않는 것

원본의 ' $0 \leq R^2 \leq 1$ '에 빠진 네 가지 주의점

$$R^2_{\text{test}} = 1 - \frac{\sum_{i \in \text{test}} (y_i - \hat{y}_i)^2}{\sum_{i \in \text{test}} (y_i - \bar{y}_{\text{test}})^2} \quad \text{음수 가능}$$

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

p = 특성 수, n = 표본 수

음수 테스트 R²는 버그가 아니라 판정이다.

주의점

- $0 \leq R^2 \leq 1$ 은 절편이 있는 OLS의 표본 내에서만 성립
- 테스트 세트에서는 음수 가능: 테스트 평균으로 찍는 것보다 못하다는 뜻
- 표본 내 R²는 특성을 더하면 절대 줄지 않는다 → 수정 R²나 테스트 세트로
- `r2_score(y_true, y_pred)`는 대칭이 아니다 — 인자 순서가 중요

03

SECTION 03

기초 실습

scikit-learn 패턴: 인스턴스화 → 적합 → 예측 → 평가

scikit-learn 4단계 패턴

선형모델이든 트리든 신경망이든 같은 네 번의 호출로 쓴다

1 인스턴스화

모델을 불러와
정의한다

```
lr =  
LinearRegression()
```

2 적합

훈련 데이터로 w 와 b
를 학습

```
lr.fit(X_train,  
      y_train)
```

3 예측

처음 보는 특성에 적용

```
y_pred =  
lr.predict(X_test)
```

4 평가

실제값과 비교

```
r2_score(y_test,  
        y_pred)
```

데이터셋 안내

- 원본 실습의 보스턴 주택가격 데이터는 scikit-learn 1.2에서 삭제되었다. 이 버전은 강의용 코스피 데이터 (섹션 06)를 쓴다.

실습 1: 단순 선형회귀

주택매매가격 하나로 코스피지수를 설명한다 (무작위 80/20 분리, seed 42)

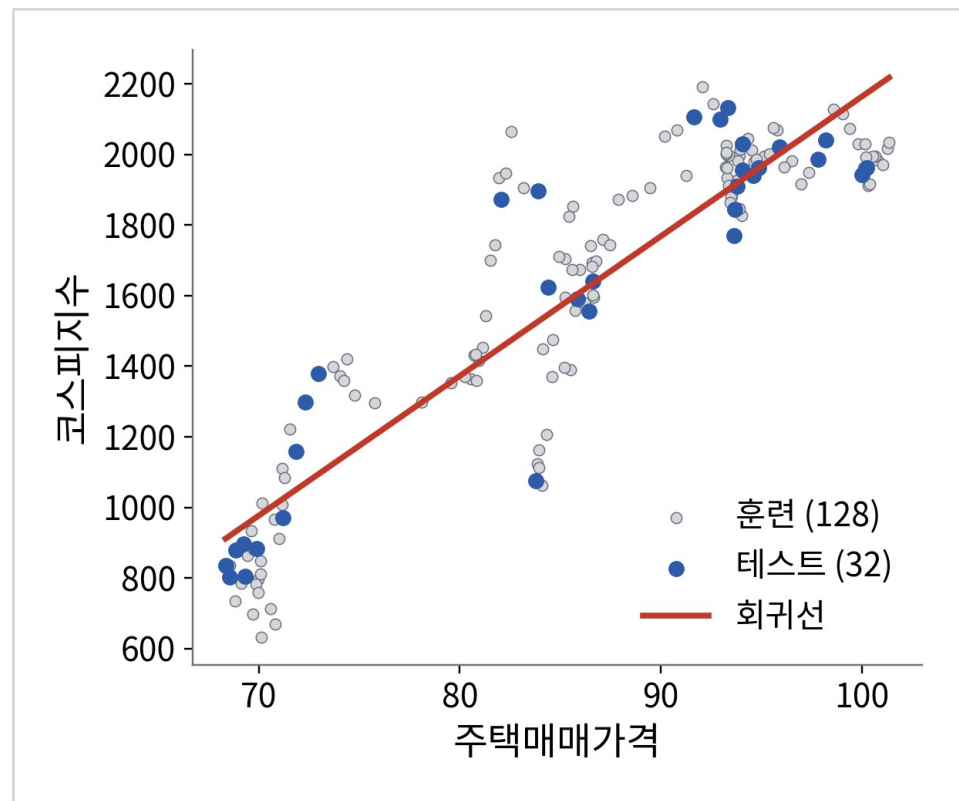
Python · Colab

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
f = ['주택매매가격']      # [[ ]]로 X를 2차원 유지

sim_lr = LinearRegression()      # 1 인스턴스화
sim_lr.fit(X_train[f], y_train)  # 2 적합
y_pred = sim_lr.predict(X_test[f]) # 3 예측
print('R2: {:.4f}'.format(r2_score(y_test, y_pred)))
print('w: {:.4f}, b: {:.4f}'.format(
    sim_lr.coef_[0], sim_lr.intercept_))
```

출력

R2: 0.8478
w: 39.5708, b: -1793.0459



지수 1포인트 상승 $\approx$ 코스피 40포인트

실습 2: 다중 선형회귀

같은 분리, 특성 29개를 모두 사용

Python · Colab

```
from sklearn.metrics import mean_squared_error
mul_lr = LinearRegression()
mul_lr.fit(X_train, y_train)          # 특성 29개
y_pred = mul_lr.predict(X_test)
print('R2: {:.4f}'.format(r2_score(y_test, y_pred)))
rmse = mean_squared_error(y_test, y_pred) ** 0.5
print('RMSE: {:.2f}'.format(rmse))
```

출력

R2: 0.9460
RMSE: 107.51

coef_에 계수 29개, intercept_ = -2905.2

모델	테스트 R ²	RMSE
단순 (특성 1개)	0.848	180.6
다중 (29개)	0.946	107.5

축배는 아직

- R²는 인자 순서를 바꾸면 값이 달라진다 (섹션 06)
- 상관된 금리 변수의 계수가 불안정하다

04

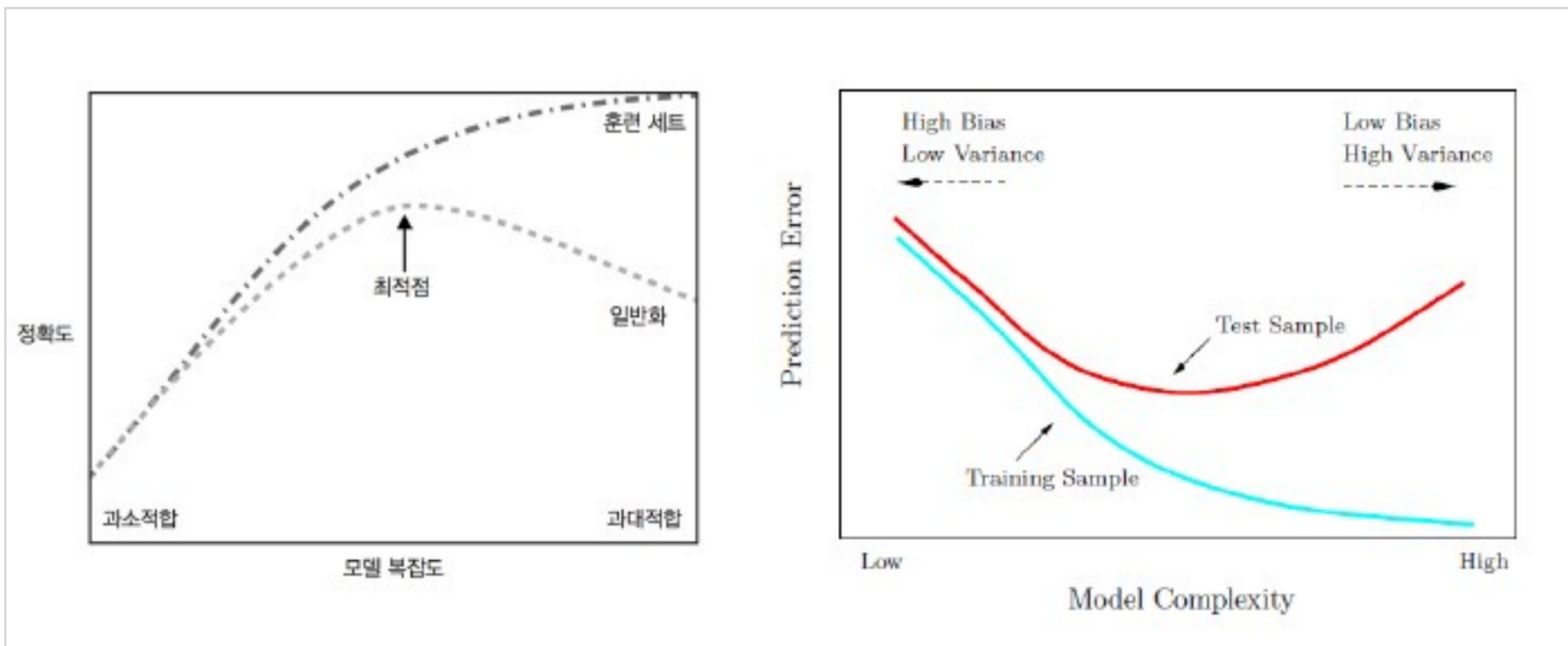
SECTION 04

규제 (Regularization)

릿지 · 라쏘 · 엘라스틱넷 — 복잡도에 가격을 매긴다

왜 규제하는가? 과대적합

특성은 많고 표본은 적으면 모델이 노이즈를 외운다: 훈련 데이터에선 잘 맞고 새 데이터에선 틀린다



과대적합 = 일반화 능력의 결여. 경고 신호: 표본 수 n 에 비해 특성 수 p 가 많다 ($p > n$ 이면 극단적).

예시: 누가 요트를 사는가?

규칙 B = 45세 초과 AND (자녀 3명 미만 OR 이혼 아님) · 데이터: Müller & Guido

나이	보유 차량	주택 보유	자녀 수	혼인 상태	애완견	보트 구매	규칙 A: 주택 보유 (9/12)	규칙 B (12/12)
66	1	yes	2	사별	no	yes	✓ yes	✓ yes
52	2	yes	3	기혼	no	yes	✓ yes	✓ yes
22	0	no	0	기혼	yes	no	✓ no	✓ no
25	1	no	1	미혼	no	no	✓ no	✓ no
44	0	no	2	이혼	yes	no	✓ no	✓ no
39	1	yes	2	기혼	yes	no	✗ yes	✓ no
26	1	no	2	미혼	no	no	✓ no	✓ no
40	3	yes	1	기혼	yes	no	✗ yes	✓ no
53	2	yes	2	이혼	no	yes	✓ yes	✓ yes
64	2	yes	3	이혼	no	no	✗ yes	✓ no
58	2	yes	2	기혼	yes	yes	✓ yes	✓ yes
33	1	no	1	미혼	no	no	✓ no	✓ no

모델 복잡도는 데이터에 맞춰야 한다

규칙 B는 12명에게 100% 맞는다 — 바로 그래서 의심해야 한다

고객이 12명일 때

- 완벽히 맞는 규칙은 얼마든지 있다 (예: '66·52·53·58세는 보트를 산다')
- 규칙 B의 세 조건은 이 12명에게 맞춘 것 → 과대적합 가능성
- 단순한 규칙 A(75%)가 새 고객에게 비슷하게 통할 수 있다

고객이 10,000명일 때

- 10,000명에게도 규칙 B가 맞다면 훨씬 더 믿을 수 있다
- 데이터가 많고 다양할수록 더 복잡한 모델을 감당한다
- 대개 데이터를 늘리는 게 정교한 튜닝보다 낫다

원본 슬라이드 수정

- 원본은 '많은 변수를 사용하는 것이 더 안정일 수 있다'고 했다. 교훈은 반대다: 복잡한 모델은 데이터가 뒷받침할 때만 안전하다.

릿지 회귀 (L2 규제)

$\lambda \geq 0$ 을 정하고 MSE에 계수 제곱합 벌점을 더한 값을 최소화한다

$$J_{\text{ridge}}(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \lambda \sum_{j=1}^n w_j^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (X'X + \lambda I)^{-1} X'y$$

λ 가 커지면 어떻게 되나?

- $\lambda = 0$: 보통의 최소제곱(OLS)
- $\lambda \uparrow$: 계수가 0 쪽으로 줄지만 정확히 0은 안 된다
- $\lambda \rightarrow \infty$: 모든 $w \rightarrow 0$, $\hat{y} \rightarrow y$ 의 평균
- 편향 $\uparrow$, 분산 $\downarrow$

실무 메모

- 먼저 표준화한다 — 벌점은 단위에 민감
- 절편 b에는 벌점을 주지 않는다
- λ 를 더하면 $X'X$ 가 역행렬을 갖는다 $\rightarrow$ 다중공선성 처방
- scikit-learn에서는 λ 를 alpha라 부른다

라쏘 회귀 (L1 규제)

릿지와 같지만 계수의 절댓값에 벌점을 준다

$$J_{\text{lasso}}(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \lambda \sum_{j=1}^n |w_j|$$

L1 벌점의 효과

- 일부 계수를 정확히 0으로 만든다
- → 덜 중요한 특성을 없앤다 (내장된 변수 선택)
- 희소한 모델 → 해석이 쉽다

실무 메모

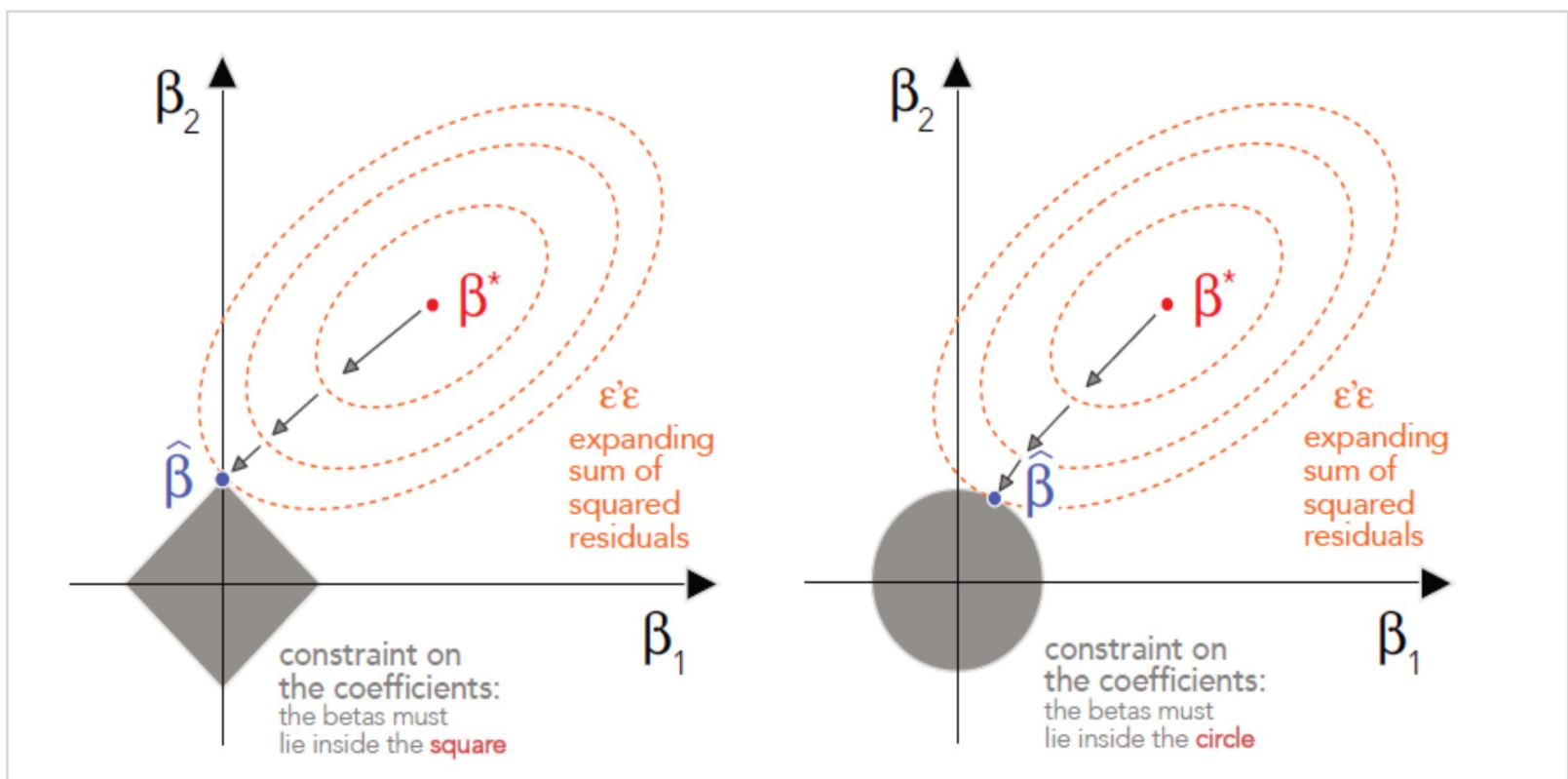
- 닫힌 해가 없다: 좌표하강법으로 푼다
- 상관된 특성 중 하나를 남기고 나머지를 버리는데, 어느 것을 남길지는 다소 임의적이다
- 먼저 표준화; $\lambda = \text{scikit-learn의 alpha}$

라쏘 대 릿지: 기하학적 해석

많은 특성 중 일부만 중요하다면 라쏘(해석이 쉽다), 여러 특성이 조금씩 기여하면 릿지

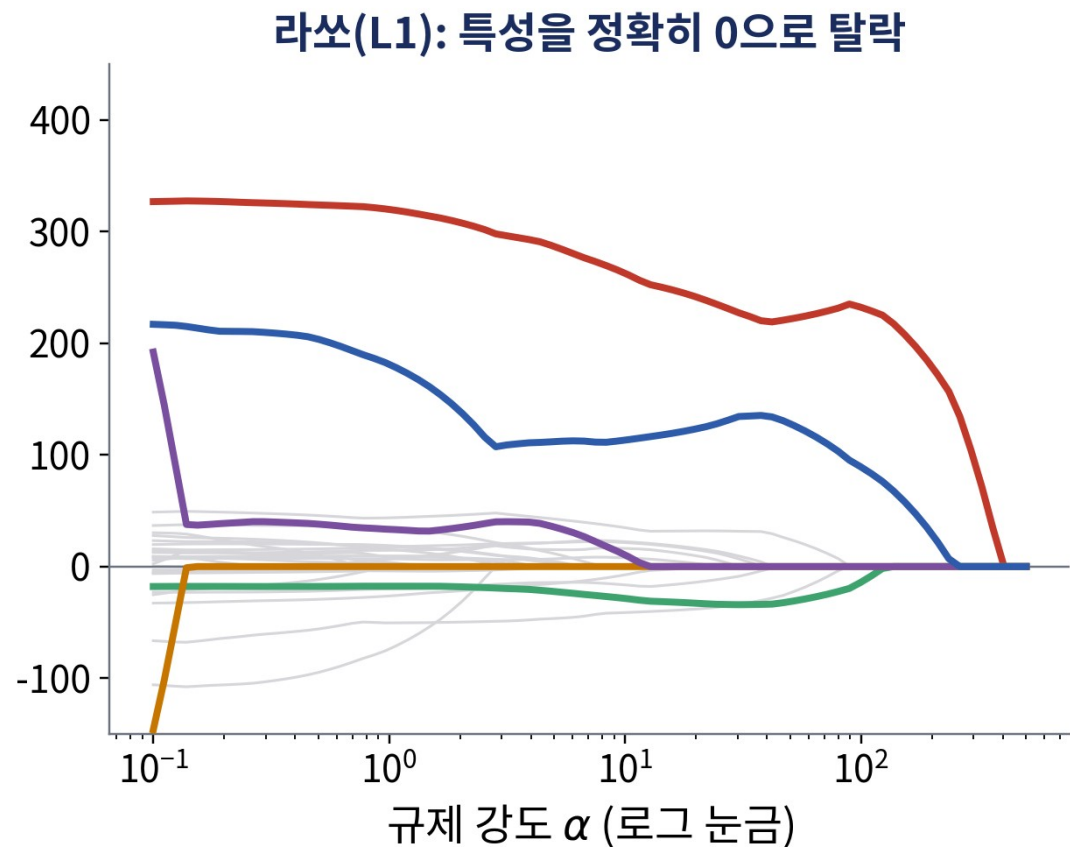
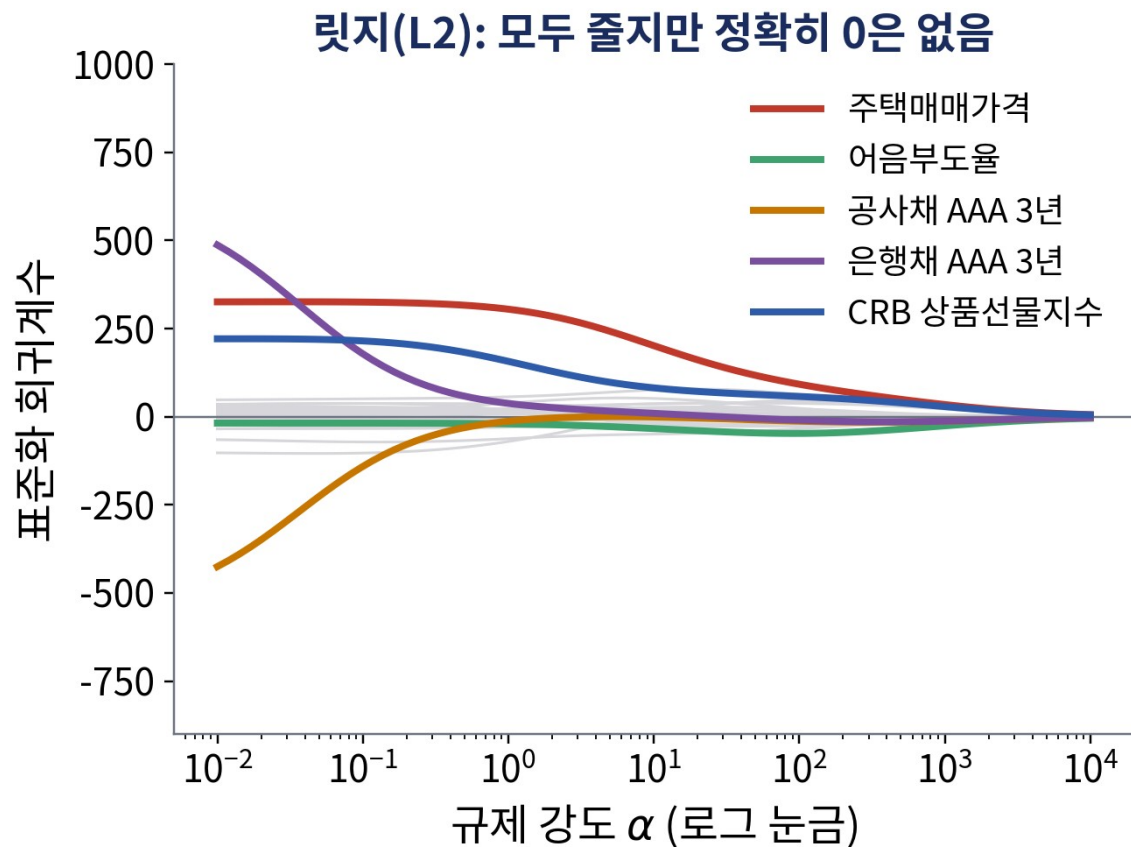
그림 읽기

- 왼쪽: 라쏘
- 오른쪽: 릿지
- 타원: 등오차선
- 꼭짓점 → 계수 0



코스피 데이터로 본 릿지 대 라쏘

α 가 커지면 릿지는 모든 계수를 부드럽게 줄이고, 라쏘는 특성을 하나씩 꺼 버린다



엘라스틱넷: 릿지 + 라쏘

두 벌점을 합친 모델 — R의 glmnet과 같은 계열 (scikit-learn 형식)

$$\frac{1}{2m} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \underbrace{\alpha \rho \|\mathbf{w}\|_1}_{\text{L1항}} + \underbrace{\frac{\alpha(1-\rho)}{2} \|\mathbf{w}\|_2^2}_{\text{L2항}}$$

$$\alpha = \lambda_1 + \lambda_2 \quad \rho = \text{l1_ratio} = \frac{\lambda_1}{\lambda_1 + \lambda_2}$$

두 개의 손잡이

- alpha: 전체 규제 강도
- l1_ratio = 1 → 라쏘, 0 → 릿지

이름 함정: R vs. Python

- glmnet: alpha = 혼합비, lambda = 강도
- sklearn: l1_ratio = 혼합비, alpha = 강도

scikit-learn의 규제 모델

셋 다 `sklearn.linear_model`에 있다 (소문자 — 파이썬은 대소문자를 구분한다)

Python · Colab

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.linear_model import ElasticNet

ridge = Ridge(alpha=1.0)           # L2
lasso = Lasso(alpha=1.0)           # L1
enet = ElasticNet(alpha=1.0, l1_ratio=0.5)

ridge.fit(X_train_std, y_train)    # 표준화된 X
```

원본에서 고친 점

- `sklearn.Linear_model` → `sklearn.linear_model`

같은 alpha ≠ 같은 강도

$$\text{Ridge : } \| \mathbf{y} - X\mathbf{w} \|_2^2 + \alpha \| \mathbf{w} \|_2^2$$

$$\text{Lasso : } \frac{1}{2m} \| \mathbf{y} - X\mathbf{w} \|_2^2 + \alpha \| \mathbf{w} \|_1$$

그래서

- 라쏘의 손실은 2m으로 나누고 릿지는 나누지 않는다
- alpha는 습관이 아니라 교차검증으로 정한다

구현 예제: 학습, 예측, 비교

원본 코드를 고쳐서 표준화한 코스피 특성에 적용 (무작위 분리, seed 42)

Python · Colab

```
from sklearn import linear_model
from sklearn.metrics import mean_squared_error
import math
ridge_model = linear_model.Ridge(alpha=1)
lasso_model = linear_model.Lasso(alpha=1)
ridge_model.fit(X_train, y_train)
lasso_model.fit(X_train, y_train)
ridge_pred = ridge_model.predict(X_test)
lasso_pred = lasso_model.predict(X_test)
ridge_rmse = math.sqrt(
    mean_squared_error(y_test, ridge_pred))
lasso_rmse = math.sqrt(
    mean_squared_error(y_test, lasso_pred))
```

출력

릿지 RMSE 107.04 · 라쏘 RMSE 103.36 (29개 중 27개 유지)

고치기 부분

- imort → import
- Ridge_modelfit → .fit
- 중복된 Lasso 줄 삭제
- sklearn_metrics → sklearn.metrics
- 빠진 ')' 추가
- 보스턴 데이터 → 코스피

05

SECTION 05

다항식 회귀모델

선형모델로 곡선을 — 그리고 뒤따르는 특성 폭증

다항식 회귀도 선형모델이다

'선형'은 계수 w 에 대해 선형이라는 뜻 — 특성 자체는 제곱하거나 곱해도 된다

$$\hat{y} = w_0 + w_1x + w_2x^2 + \cdots + w_dx^d$$

$$(x_1, x_2, x_3) \xrightarrow{\text{2차 전개}} (1, x_1, x_2, x_3, x_1^2, x_2^2, x_3^2, \underbrace{x_1x_2, x_1x_3, x_2x_3}_{\text{상호작용항}})$$

상호작용항 (interaction term)

- $x_1 \cdot x_2$: x_1 의 효과가 x_2 에 따라 달라지게 한다
- 실무에서 가장 쓸모 있는 새 항인 경우가 많다

$$\text{열 개수} = \binom{p+d}{d} = \binom{29+2}{2} = 465$$

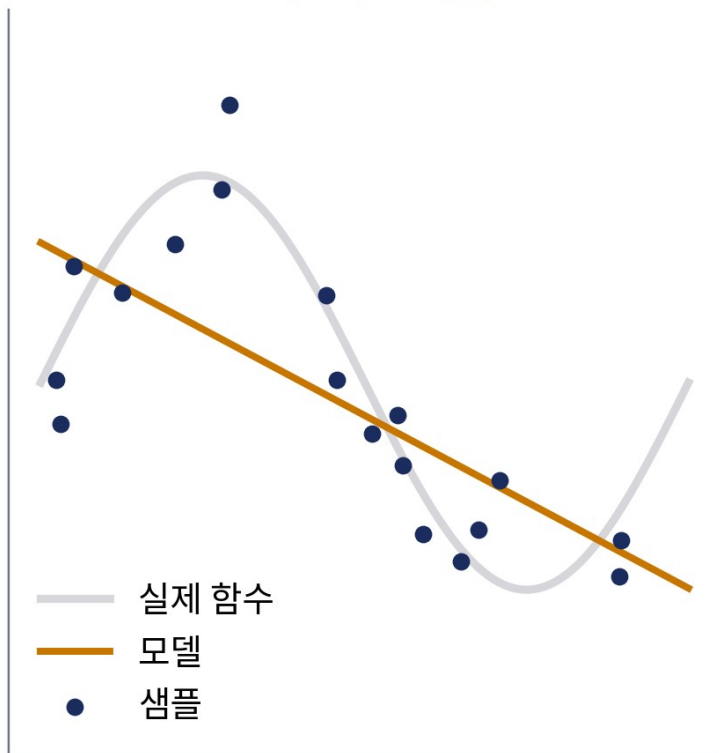
열 465개 vs. 훈련 행 128개

- $p > n \rightarrow$ 규제 필수, 보통 2~3차면 충분

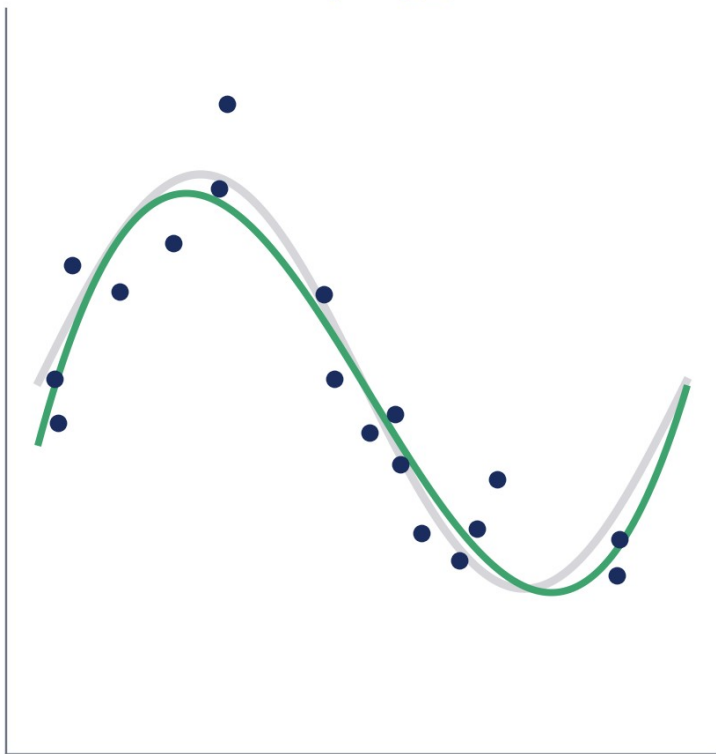
차수가 복잡도를 정한다

1차는 과소적합, 3차는 곡선을 잡고, 12차는 점 사이의 노이즈까지 쫓아간다

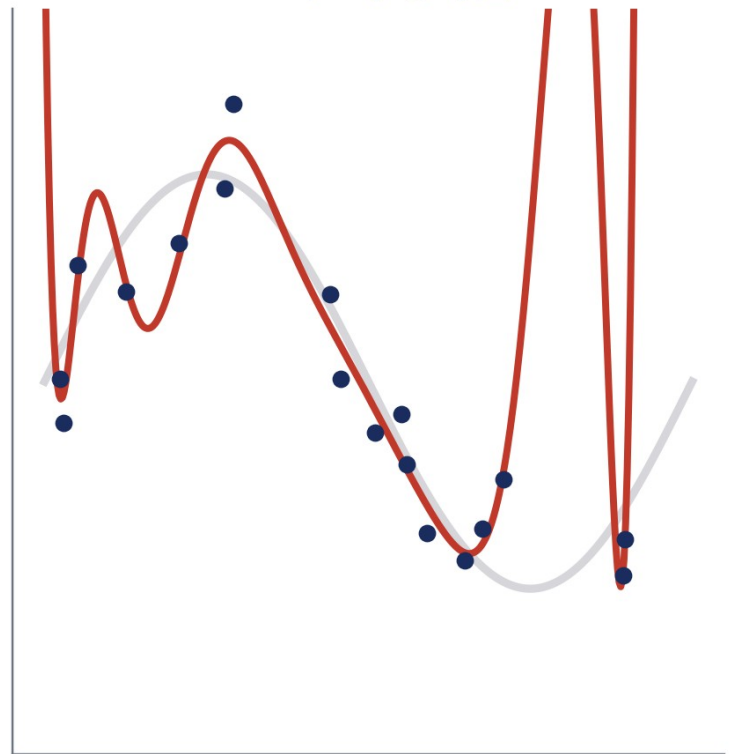
1차: 과소적합



3차: 적절



12차: 과대적합



scikit-learn의 PolynomialFeatures

모델이 아니라 변환기: 훈련 데이터로 전개를 fit하고, 테스트 데이터에는 그대로 적용한다

Python · Colab

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2,
                          interaction_only=False,
                          include_bias=True)

poly_X_train = poly.fit_transform(X_train)
poly_X_test = poly.transform(X_test) # fit 금지!
print(poly_X_train.shape)           # (128, 465)
```

이후 어떤 선형모델이든

```
lr = LinearRegression().fit(poly_X_train, y_train)
ridge = Ridge(alpha=1).fit(poly_X_train, y_train)
```

파라미터

- `interaction_only=True`: 곱항만, 제곱항 없음
- `include_bias`는 1의 열을 추가 — 절편을 따로 적합하는 `LinearRegression`과 쓸 때는 `False`
- 오타 수정: `PolynomilaFeature` → `PolynomialFeatures`

06

SECTION 06

코스피 사례 실습

Colab 노트북에서: 160개월, 지표 29개, 지수 하나

데이터: 지표 29개, 160개월

월별, 2003년 5월 ~ 2016년 8월 · 목표변수 = 코스피 월말 종가 · 파일: KOSPI_Index_KO.csv

그룹 (개수)	변수
심리지표 (3)	전경련BSI, 건설BSI(전망), 한국(OECD) 경기선행지표
실물경제 · 신용 (5)	주택매매가격, 실업률, 어음부도율, 내국인출국자수 (수준, 전년동월비)
교역 (1)	일별수출액 전년동월비
금리 · 스프레드 (7)	통안채 364일, 공사채 · 은행채 AAA 3년, 장단기 스프레드 2개, 신용 스프레드 2개
시장위험 (1)	코스피 200 변동성지수(VKOSPI)
국민계정 (8)	분기 값을 월별 반복: GDP 서비스업, 설비 · 건설투자, 재화수출, 내수, 디플레이터, 총저축률 · 총투자율
원자재 (4)	로이터/CRB 동일가중: 상품선물, 에너지, 산업, 귀금속

원본 파일에는 날짜가 없어 코스피 월말 종가와 대조해 추정했다 (예: 2008년 10월 = 1,113, 2016년 8월 = 2,035).

노트북 파이프라인

CSV에서 적합한 모델 6개까지 다섯 단계

1 불러오기

CSV 읽기

```
pd.read_csv(...)
```

2 X / y

특성과 목표변수

```
X = df.drop(...)
```

3 분리

80 / 20

```
train_test_split
```

4 표준화

훈련에서만 fit

```
StandardScaler
```

5 모델

OLS, 릿지, 라쏘,
다항식

```
fit → predict
```

원본 노트북 코드

```
std = StandardScaler()  
X_train = std.fit_transform(X_train)  
X_test = std.transform(X_test) # 여기서 fit하면 안 됨
```

올바른 감각

- 테스트 데이터가 적합에 새어 들면 안 된다

노트북 결과 (무작위 분리)

모든 숫자를 재현했다. 'R² 출력값' 열은 $r^2(y_pred, y_test) \times 100$ 의 결과

모델	테스트 RMSE	R ² 출력값	R ² 올바른 값	훈련 R ²	0 아닌 계수
선형회귀	107.5	94.6	0.946	0.968	29
릿지 $\alpha = 100$	159.3	82.7	0.882	0.909	29
라쏘 $\alpha = 100$	173.5	73.6	0.860	0.835	3
2차 다항식 + OLS	129.1	91.9	0.922	1.000	464
2차 다항식 + 라쏘 $\alpha = 100$	169.7	77.2	0.866	0.850	7
2차 다항식 + 릿지 $\alpha = 1$	124.7	92.5	0.927	1.000	464

라쏘 $\alpha = 100$ 이 남긴 특성은 셋뿐: 주택매매가격(+222), CRB 상품선물지수(+86), 어음부도율(-23)

함정 1: r2_score는 대칭이 아니다

함수 형식은 `r2_score(y_true, y_pred)` — 순서를 바꾸면 분모가 달라진다

✗ 원본 노트북

```
r2(y_pred, y_test) * 100
```

✓ 올바른 방식

```
r2_score(y_test, y_pred)
```

모델	순서 바뀐	올바른 값
릿지 $\alpha = 100$	0.827	0.882
라쏘 $\alpha = 100$	0.736	0.860
2차 다항식 + 라쏘	0.772	0.866

이유

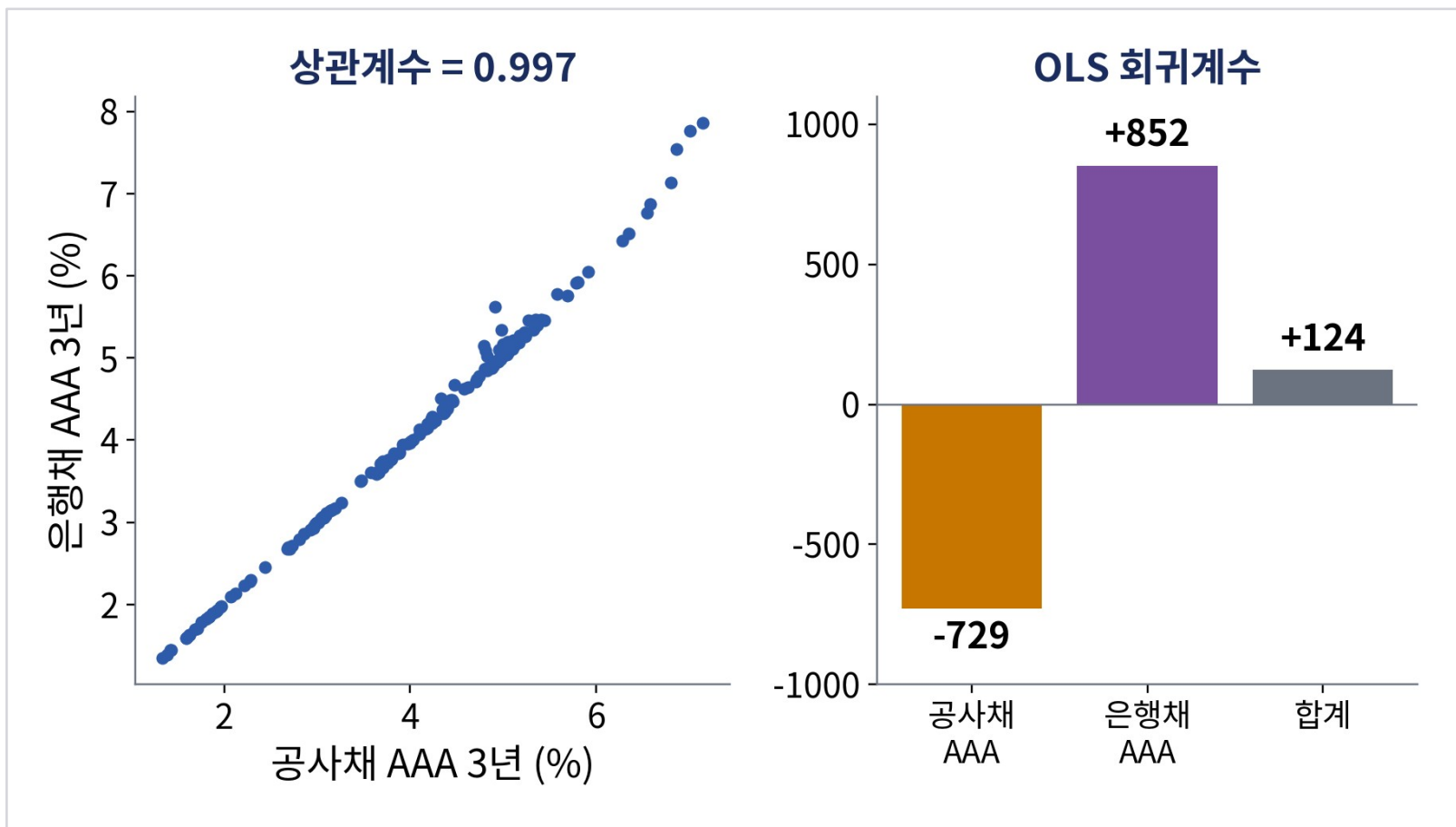
- R^2 는 첫 번째 인자의 분산으로 나눈다
- 순서를 바꾸면 예측값의 분산으로 나누게 된다
- RMSE는 영향 없음: 제곱오차는 대칭

함정 2: 공선성 높은 금리, 폭발하는 계수

AAA 3년물 금리 두 개가 함께 움직여 OLS가 크고 부호가 반대인 계수를 준다

진단

- VIF 각각 약 2,000
- 순효과 +124뿐
- 부호 해석 불가
- 대책: 릿지



토론 질문

Q1

노트북은 OLS의 R^2 를 94.6%로 보고한다. 테스트는 32개월뿐인데, 무작위 분할을 바꾸면 이 숫자는 얼마나 흔들릴까?

평가**Q2**

라쏘는 주택가격, CRB 원자재, 어음부도율을 남겼다. 이 변수들이 코스피를 움직인다고 말할 수 있나?

해석**Q3**

2차 특성은 128행에 465열이다. 그런데도 릿지($\alpha = 1$)가 유일한 해를 주는 이유는?

규제**Q4**

OLS의 테스트 RMSE 107.5는 테스트 세트 코스피 평균의 약 6.8%다. 이 오차는 쓸 만한 크기인가?

척도

요약

1

OLS는 제곱오차를 최소화한다

정규방정식 또는 경사하강법. 다중공선성($X'X$ 가 특이행렬에 가까움)을 경계한다.

2

표본 밖에서, 정직하게 평가한다

모델이 보지 못한 테스트 세트에서 RMSE · MAE · R^2 를 잔다.

3

규제로 복잡도를 데이터에 맞춘다

릿지는 줄이고, 라쏘는 고르고, 엘라스틱넷은 둘 다. α 는 교차검증으로 정한다.

과제: 한국어 Colab 노트북을 실행해 섹션 06의 숫자를 모두 재현해 보라.

원본 덱 정오표 (1/2)

슬라이드 번호는 원본 한국어 덱 기준

원본 슬라이드	원본	수정
5	$\hat{y} = b_0x_0 + b_1x_1 + \dots + b_nx_n + b_0$ (절편 중복)	$x_0 = 1$ 로 둔 $\hat{y} = \sum w_j x_j$, 또는 $\hat{y} = w'x + b$ — 둘 중 하나만
6	$J = 1/(2m) \sum (y - \hat{y})^2$ 를 'MSE'로 표기	MSE는 $1/m$. $1/2$ 은 미분 편의용이며 최적해는 같다
7	X 구성요소 간 상관이 크면 '분포'가 0에 가까워진다	$X'X$ 의 행렬식이 0에 가까워져 $\hat{w}$ 의 분산이 폭발한다
11	RMSE를 '대략 평균 오차'로 해석	$RMSE \geq MAE$. 전형적 절대 오차는 MAE. MAPE는 $y \approx 0$ 에서 불안정
12	R^2 는 0과 1 사이 / '환전하게' 오타	절편 있는 OLS의 표본 내에서만. 테스트 R^2 는 음수 가능, 수정 R^2 추가
18-19	'45세 이상, 자녀 셋 미만, 이혼하지 않은' / '많은 변수가 더 안정'	규칙은 45세 초과 AND (자녀 3명 미만 OR 이혼 아님). 교훈: 복잡한 규칙엔 더 많은 데이터

원본 덱 정오표 (2/2)

슬라이드 번호는 원본 한국어 덱 기준

원본 슬라이드	원본	수정
20	'파라미터 를 선택해'(λ 누락), 질문만 있고 답 없음	λ 복원. 답: 계수 축소, 편향 $\uparrow$ 분산 $\downarrow$, λ 가 특이성 해소
23	엘라스틱넷 손실을 MSE로 표기	scikit-learn은 $(1/2m) \cdot \text{RSS}$. glmnet은 $\alpha \cdot \lambda$ 의 의미가 반대
24	from sklearn.Linear_model import ...	sklearn.linear_model. 릿지와 라쏘는 α 척도가 다르다
25	imort, Ridge_modelfit, sklearn_metrics, 중복 줄, ')' 누락	모두 수정. 보스턴 데이터(scikit-learn 1.2에서 삭제) $\rightarrow$ 코스피
27-28	'1차원 변수를 2차원 이상으로' / PolynomilaFeature(s) / '훈련세'	차원이 아니라 차수(degree). PolynomialFeatures. include_bias 설명 추가
노트북	$r2(y_pred, y_test)$ — 인자 순서가 바뀜	$r2_score(y_test, y_pred)$